

DevOps Monitoring Phase

PPT Content

Slide 1: Title

DevOps Lifecycle: Monitoring Phase

Continuous Monitoring for Better Performance, Reliability & Availability

Presented by: Harsh Maheshwari

Slide 2: What is DevOps Monitoring?

Definition

DevOps Monitoring is the continuous process of observing applications, servers, infrastructure, and services to:

- Measure performance
- Detect problems
- Generate alerts
- Maintain availability
- Improve system reliability

Simple Idea

Deploy → Monitor → Detect → Fix → Improve

Slide 3: Monitoring in DevOps Lifecycle

DevOps Lifecycle

Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → Feedback

Monitoring provides feedback to the development and operations teams.

Key Point

DevOps does not end after deployment. Continuous monitoring helps improve the system continuously.

Slide 4: Why is Monitoring Important?

Without Monitoring

- Problems may remain unnoticed
- Users experience failures
- Downtime increases
- Troubleshooting takes longer

With Monitoring

- Problems are detected early
 - Teams receive alerts
 - Performance can be analyzed
 - Downtime can be reduced
 - System reliability improves
-

Slide 5: What Do We Monitor?

Application

- Response time
- Error rate
- Availability
- Requests

Server

- CPU
- Memory
- Disk
- Network

Database

- Query performance
- Connections
- Storage
- Errors

Network

- Traffic
- Latency
- Packet loss

Slide 6: Important Monitoring Metrics

Metric	What it tells us
CPU Usage	Processor utilization
Memory Usage	RAM utilization
Disk Usage	Storage utilization
Response Time	Application speed
Error Rate	Failed requests
Uptime	Service availability
Network Traffic	Data transmission
Latency	Communication delay

Slide 7: Metrics vs Logs

Metrics

Numerical measurements of system performance.

Examples:

- CPU = 75%
- Memory = 68%
- Error Rate = 2%
- Response Time = 200 ms

Logs

Records of events that occurred in a system.

Example:

```
10:30 User Login Successful
10:31 Database Connected
10:32 Payment Failed
```

Remember

Metrics tell us WHAT is happening.

Logs help us understand WHAT HAPPENED.

Slide 8: Alerts & Thresholds

What is an Alert?

An alert is a notification generated when a monitoring condition is met.

Example

```
CPU Usage
  ↓
65% → Normal
  ↓
80% → Warning
  ↓
90% → Critical
  ↓
Alert Generated
```

Alerts can notify the DevOps team through:

- Email
- Messaging systems
- Monitoring dashboards
- Incident management systems

Slide 9: Monitoring Tools

Popular DevOps Monitoring Tools

Prometheus

- Collects and stores metrics
- Supports alerting

Grafana

- Creates dashboards
- Visualizes monitoring data

ELK Stack

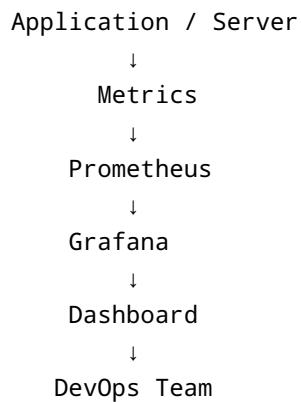
- Log collection and analysis
- Elasticsearch + Logstash + Kibana

Other Tools

- Nagios
 - Datadog
 - New Relic
-

Slide 10: Prometheus + Grafana

How They Work Together



Prometheus

Collects and stores metrics

Grafana

Visualizes metrics using dashboards

Slide 11: Real-World Example

E-Commerce Website

Suppose an online shopping website suddenly becomes slow.

Monitoring shows:

CPU Usage	→ 95%
Memory Usage	→ 90%
Response Time	→ 5 seconds
Error Rate	→ 12%

What happens?

```
graph TD; A[Problem Detected] --> B[Alert Generated]; B --> C[DevOps Team Notified]; C --> D[Problem Investigated]; D --> E[Issue Fixed]; E --> F[System Monitored Again];
```

Result

Reduced downtime + Better user experience

Slide 12: Benefits & Conclusion

Benefits of Monitoring

- Early problem detection
- Reduced downtime
- Better performance
- Faster troubleshooting
- Improved reliability
- Better resource utilization
- Improved user experience

Conclusion

Monitoring creates a continuous feedback loop that helps DevOps teams detect problems, improve performance, and maintain reliable applications.

Thank You

Questions?